

Name of Student: Aryyan Paygude	PRN:25070122291
Semester: III	Year : AY 2026-27
Subject: Programming with Java	
ASSIGNMENT No : 3	
TITLE : Method Overloading and Static Members	

Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

Programme

```
class Calculator {
    static int calculationCount = 0;    // static variable shared by all objects

    // overloaded methods - same name, different parameter lists
    int add(int a, int b) {
        calculationCount++;
        return a + b;
    }

    double add(double a, double b) {
        calculationCount++;
        return a + b;
    }

    int add(int a, int b, int c) {
        calculationCount++;
        return a + b + c;
    }

    static void showCount() {
        System.out.println("Total calculations performed: " + calculationCount);
    }
}

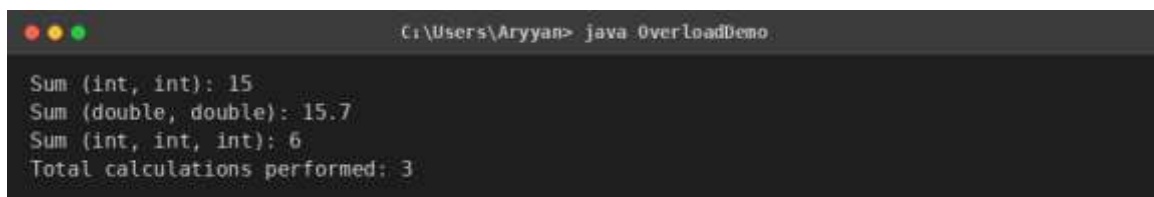
public class OverloadDemo {
    public static void main(String[] args) {
```

```
Calculator calc = new Calculator();

System.out.println("Sum (int, int): " + calc.add(5, 10));
System.out.println("Sum (double, double): " + calc.add(5.5, 10.2));
System.out.println("Sum (int, int, int): " + calc.add(1, 2, 3));

Calculator.showCount();
}
}
```

Output

A screenshot of a Java IDE terminal window. The title bar shows the file path 'C:\Users\Aryyan> java OverloadDemo'. The terminal output displays four lines: 'Sum (int, int): 15', 'Sum (double, double): 15.7', 'Sum (int, int, int): 6', and 'Total calculations performed: 3'.

```
C:\Users\Aryyan> java OverloadDemo
Sum (int, int): 15
Sum (double, double): 15.7
Sum (int, int, int): 6
Total calculations performed: 3
```

Conclusion

Thus, a Java program was developed to demonstrate method overloading and the use of static variables and methods. It was observed that overloaded methods with the same name but different parameter lists allow the same operation (addition) to work with different data types and numbers of arguments, resolved at compile time. The static variable `calculationCount` was shared across all method calls, correctly tracking the total number of operations performed, demonstrating that static members belong to the class rather than any individual object.